

Wydział Informatyki Politechniki Białostockiej

Systemy wbudowane

Tytuł: Prosty procesor

Wykonujący: Łukasz Modzelewski, Mariusz Rostkowski

PS 1

Zespół 4

Praca laboratoryjna 11

Studia dzienne

Informatyka

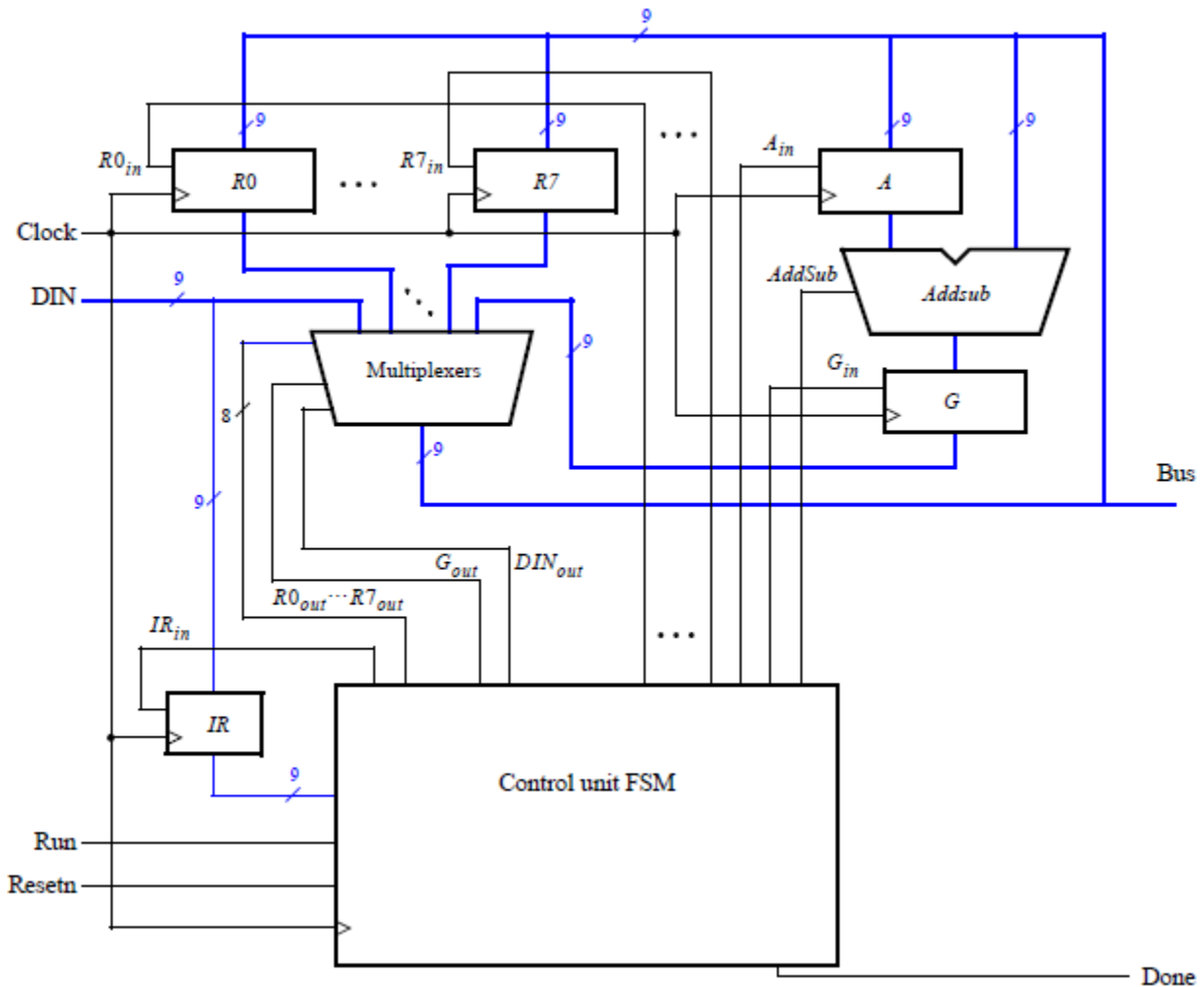
Semestr VI

Prowadzący: prof. dr hab. inż. Valery Salauyou

Data: 26.05.2022

Zadanie 1

Utwórz projekt proc_1 prostego procesora ze strukturą z rys. 1.



Rys. 1. Struktura prostego procesora

Zestaw instrukcji procesora pokazano w tabeli. 1.

Tabela. 1. Instrukcje w procesorze

Operation	Function performed
mv Rx, Ry	$Rx \leftarrow [Ry]$
mvi $Rx, \#D$	$Rx \leftarrow D$
add Rx, Ry	$Rx \leftarrow [Rx] + [Ry]$
sub Rx, Ry	$Rx \leftarrow [Rx] - [Ry]$

Tutaj $[Rx]$ oznacza zawartość rejestru Rx , $\leftarrow$ – przekazywanie danych.

Komenda mv (move) ładuje zawartość rejestru Ry do rejestru Rx, komenda mvi (move immediate) ładuje stałą D do rejestru Rx, polecenie add dodaje zawartość rejestrów Rx i Ry, wynik jest zapisywany do rejestru Rx; polecenie sub od zawartości rejestru Rx odejmuje zawartość rejestru Ry, wynik jest zapisany w Rx.

Każda instrukcja procesora przedstawiona jest w formacie IIIXXYYYY, gdzie

III - kod polecenia;

XXX to rejestr Rx;

YYY jest kodem rejestru Ry.

Dla naszego procesora akceptowane są następujące kody poleceń:

mv - 000

mvi - 001

add - 010

sub - 011

Dodatkowy bit w kodzie polecenia pozwala rozszerzyć listę poleceń w przyszłości.

Polecenia mv i mvi są wykonywane w jednym cyklu zegara, polecenia add i sub wykonywane są w trzech cyklach zegara. Sygnały sterujące ustawione na 1 przy każdym takcie zegara dla każdego polecenia są pokazane w tabeli. 2.

Tabela. 2. Sygnały sterujące ustawione w każdej instrukcji / cyklu

	T_1	T_2	T_3
(mv): I_0	$RY_{out}, RX_{in},$ $Done$		
(mvi): I_1	$DIN_{out}, RX_{in},$ $Done$		
(add): I_2	RX_{out}, A_{in}	RY_{out}, G_{in}	$G_{out}, RX_{in},$ $Done$
(sub): I_3	RX_{out}, A_{in}	$RY_{out}, G_{in},$ $AddSub$	$G_{out}, RX_{in},$ $Done$

Wskazówka: szkielet kodu procesora pokazano na rys. 2, a, b.

```
module proc (DIN, Resetn, Clock, Run, Done, BusWires);
```

```
    input [8:0] DIN;
```

```
    input Resetn, Clock, Run;
```

```
    output Done;
```

```
    output [8:0] BusWires;
```

```
    parameter T0 = 2'b00, T1 = 2'b01, T2 = 2'b10, T3 = 2'b11;
```

```
    ... deklaracja zmiennych
```

```
    assign I = IR[1:3];
```

```
    dec3to8 decX (IR[4:6], 1'b1, Xreg);
```

```
    dec3to8 decY (IR[7:9], 1'b1, Yreg);
```

Rys. 2, a. Szkielet kodu Verilog dla procesora

```
// Zarządzaj tabelą stanów FSM
always @(Tstep_Q, Run, Done)
begin
    case (Tstep_Q)
        T0: // W tej chwili dane są ładowane do IR
            if (!Run) Tstep_D = T0;
            else Tstep_D = T1;
        T1....
    endcase
end

// Sterowanie wejściami FSM
always @(Tstep_Q or I or Xreg or Yreg)
begin
    ... określenie wartości początkowych
    case (Tstep_Q)
        T0: // zapamiętaj DIN w IR w takcie 0
            begin
                IRin = 1'b1;
            end
        T1: // określ sygnały w takcie 1
            case (I)
                ...
            endcase
        T2: // określ sygnały w takcie 2
            case (I)
                ...
            endcase
        T3: // określ sygnały w takcie 3
            case (I)
                ...
            endcase
    endcase
end
```

```
// sterowanie przerzutnikami FSM
always @(posedge Clock, negedge Resetn)
    if (!Resetn)
        ...

regn reg_0 (BusWires, Rin[0], Clock, R0);
... Inicjuj inne rejestry i dodaj moduł sumatora / odejmowania

... określ szynę
```

Rys. 2,b. Szkielet kodu **Verilog** procesora

Opis zastosowanych modułów pokazano na rys. 2, c.

```
module dec3to8(W, En, Y);
    input [2:0] W;
    input En;
    output [0:7] Y;
    reg [0:7] Y;

    always @(W or En)
    begin
        if (En == 1)
            case (W)
                3'b000: Y = 8'b10000000;
                3'b001: Y = 8'b01000000;
                3'b010: Y = 8'b00100000;
                3'b011: Y = 8'b00010000;
                3'b100: Y = 8'b00001000;
                3'b101: Y = 8'b00000100;
                3'b110: Y = 8'b00000010;
                3'b111: Y = 8'b00000001;
            endcase
        else
            Y = 8'b00000000;
        end
    end
endmodule
```

```
module regn(R, Rin, Clock, Q);
```

```
    parameter n = 9;
```

```
    input [n-1:0] R;
```

```
    input Rin, Clock;
```

```
    output [n-1:0] Q;
```

```
    reg [n-1:0] Q;
```

```
    always @(posedge Clock)
```

```
        if (Rin)
```

```
            Q <= R;
```

```
endmodule
```

Rys. 2, c. Moduły podukładów do wykorzystania w procesorze

Wykonaj symulację działania procesora dla kodu:

```
mvi R0, #D    // 30 ns
```

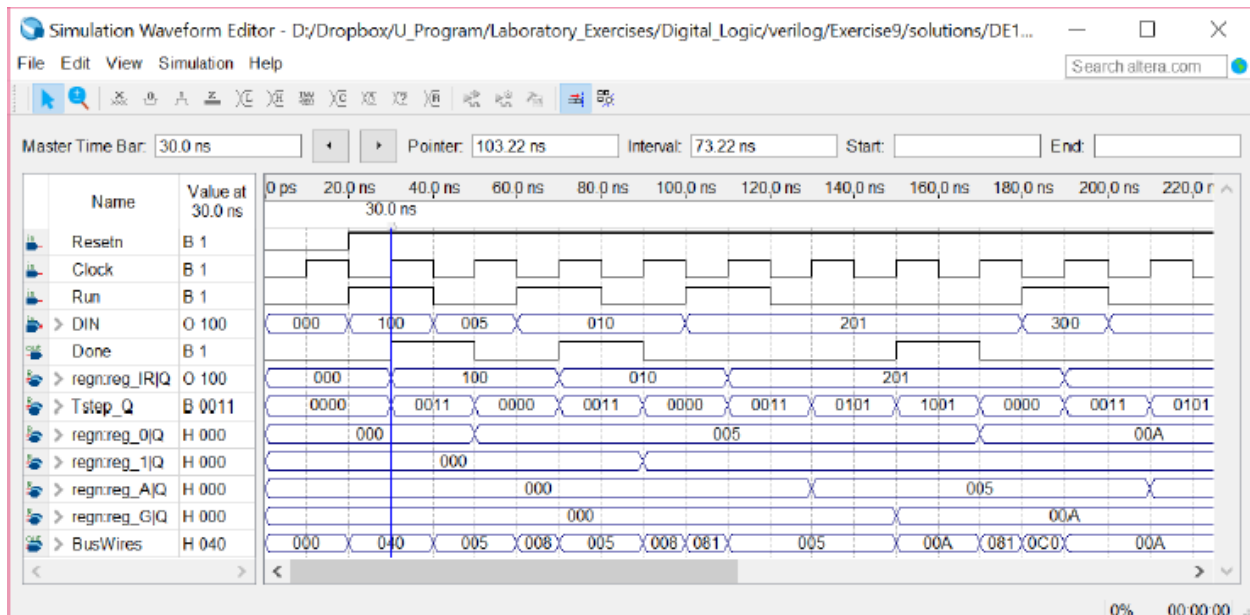
```
D = 5        // 50 ns
```

```
mv R1, R0    // 90 ns
```

```
add R0, R1   // 110 ns
```

```
sub R0, R0   // 190 ns
```

Wyniki symulacji powinny pokrywać się z wynikami przedstawionymi na rys. 3.



Rys. 3. Wyniki symulacji procesora

Kod w Verilog

Moduł procesora

```
module proc_1(
    input [8:0] DIN,
    input Resetn, clock, Run,
    output reg Done,
    output reg [8:0] Buswires);

    // stany FSM
    localparam [1:0] T0 = 2'b00, T1 = 2'b01, T2 = 2'b10, T3 = 2'b11;
    // kody instrukcji
    localparam [1:0] mv = 2'b00, mvi = 2'b01, add = 2'b10, sub = 2'b11;
    wire [0:7] Xreg, Yreg;
    wire [2:0] I;
    wire [9:1] IR;
    wire [8:0] A, G;
    wire [8:0] R0, R1, R2, R3, R4, R5, R6, R7;
    reg G_in, G_out, DIN_out, AddSub, IR_in, A_in;
    wire [8:0] Sum;
    reg [1:0] Tstep_Q /* synthesis keep */;
    reg [1:0] Tstep_D;
    reg [0:7] R_in, R_out;

    assign I = IR[9:7];
    dec3to8 decx(IR[6:4], 1'b1, Xreg);
    dec3to8 decy(IR[3:1], 1'b1, Yreg);

    // sterowanie przejściami między stanami FSM
    always @ (Tstep_Q, Run, Done)
        case (Tstep_Q)
            T0:
                if (~Run) Tstep_D = T0;
                else Tstep_D = T1;
            T1:
                if (Done) Tstep_D = T0;
                else Tstep_D = T2;
            T2:
                Tstep_D = T3;
            T3:
                Tstep_D = T0;
        endcase

    // sterowanie wyjściami FSM
    always @ (Tstep_Q, I, Xreg, Yreg) begin
        Done = 1'b0;
        G_in = 1'b0;
        G_out = 1'b0;
        A_in = 1'b0;
        AddSub = 1'b0;
        DIN_out = 1'b0;
        R_in = 8'b0000_0000;
        R_out = 8'b0000_0000;
        IR_in = 1'b0;
        case (Tstep_Q)
            T0: // zapamiętujemy DIN w IR w takcie 0
                IR_in = 1'b1;
            T1: // wyjścia z FSM w takcie 1
                case (I)
                    mv: begin
                        R_out = Yreg;
                        R_in = Xreg;
                        Done = 1'b1;
                    end
                    mvi: begin
                        DIN_out = 1'b1;
                        R_in = Xreg;
                        Done = 1'b1;
                    end
                    add, sub: begin
                        R_out = Xreg;
                        A_in = 1'b1;
                    end
                endcase
            T2: // wyjścia z FSM w takcie 2
                case (I)
                    // mv, mvi: ; // nie występują
                    add: begin
                        R_out = Yreg;
                        G_in = 1'b1;
                    end
                    sub: begin
                        R_out = Yreg;
                        G_in = 1'b1;
                        AddSub = 1'b1;
                    end
                endcase
            T3: // wyjścia z FSM w takcie 3
                case (I)
                    // mv, mvi: ; // nie występują
                    add, sub: begin
                        G_out = 1'b1;
                        R_in = Xreg;
                    end
                endcase
        endcase
    end
```

```

        Done = 1'b1;
    end
endcase
endcase
end

// sterowanie przerzutnikami FSM
always @ (posedge clock, negedge Resetn)
    if (~Resetn) Tstep_Q <= T0;
    else Tstep_Q <= Tstep_D;

// rejestry
regn #(9) reg_0(Buswires, R_in[0], clock, R0);
regn #(9) reg_1(Buswires, R_in[1], clock, R1);
regn #(9) reg_2(Buswires, R_in[2], clock, R2);
regn #(9) reg_3(Buswires, R_in[3], clock, R3);
regn #(9) reg_4(Buswires, R_in[4], clock, R4);
regn #(9) reg_5(Buswires, R_in[5], clock, R5);
regn #(9) reg_6(Buswires, R_in[6], clock, R6);
regn #(9) reg_7(Buswires, R_in[7], clock, R7);
regn #(9) reg_A(Buswires, A_in, clock, A);
regn #(9) reg_G(Sum, G_in, clock, G);
regn #(9) reg_IR(DIN[8:0], IR_in, clock, IR);

// alu
ripple_carry_adder_subtractor #(9) alu(A, Buswires, AddSub, Sum);

// multiplexer
wire [0:9] mux_s;
assign mux_s = {DIN_out, R_out, G_out};
always @ (*)
    case (mux_s)
        default: Buswires = DIN;
        10'b0100000000: Buswires = R0;
        10'b0010000000: Buswires = R1;
        10'b0001000000: Buswires = R2;
        10'b0000100000: Buswires = R3;
        10'b0000010000: Buswires = R4;
        10'b0000001000: Buswires = R5;
        10'b0000000100: Buswires = R6;
        10'b0000000010: Buswires = R7;
        10'b0000000001: Buswires = G;
    endcase
endmodule

```

Moduł N-bitowego rejestru

```

module regn #(parameter N = 9)(
    input [N-1:0] R,
    input Rin, clock,
    output reg [N-1:0] Q);

    always @ (posedge clock)
        if (Rin) Q <= R;
        else Q <= Q;

endmodule

```

Moduł dekodera z kodu binarnego na 8-bitowy kod one-hot

```

module dec3to8(
    input [2:0] w,
    input En,
    output reg [0:7] Y);

    always @ (w, En)
        if (En == 1)
            case (w)
                3'b000: Y = 8'b10000000;
                3'b001: Y = 8'b01000000;
                3'b010: Y = 8'b00100000;
                3'b011: Y = 8'b00010000;
                3'b100: Y = 8'b00001000;
                3'b101: Y = 8'b00000100;
                3'b110: Y = 8'b00000010;
                3'b111: Y = 8'b00000001;
            endcase
        else
            Y = 8'b00000000;

endmodule

```

Moduł sumatora-subtraktora, który używamy jako jednostka arytmetyczno-logiczna (ALU)

```
module ripple_carry_adder_subtractor
  #(parameter N = 4)(
    input [N-1:0] A, B,
    input sub,
    output [N-1:0] S);
  // output cout,
  // output overflow);

  wire [N-2:0] c;
  // assign overflow = c[N-2] ^ cout;
  generate
    genvar i;
    for (i = 0; i < N; i = i + 1)
      begin: ad
        case (i)
          0: full_adder fa(A[i], sub ^ B[i], sub, S[i], c[i]);
          N-1: full_adder fa(A[i], sub ^ B[i], c[i-1], S[i], /*cout*/);
          default: full_adder fa(A[i], sub ^ B[i], c[i-1], S[i], c[i]);
        endcase
      end
    endgenerate
endmodule
```

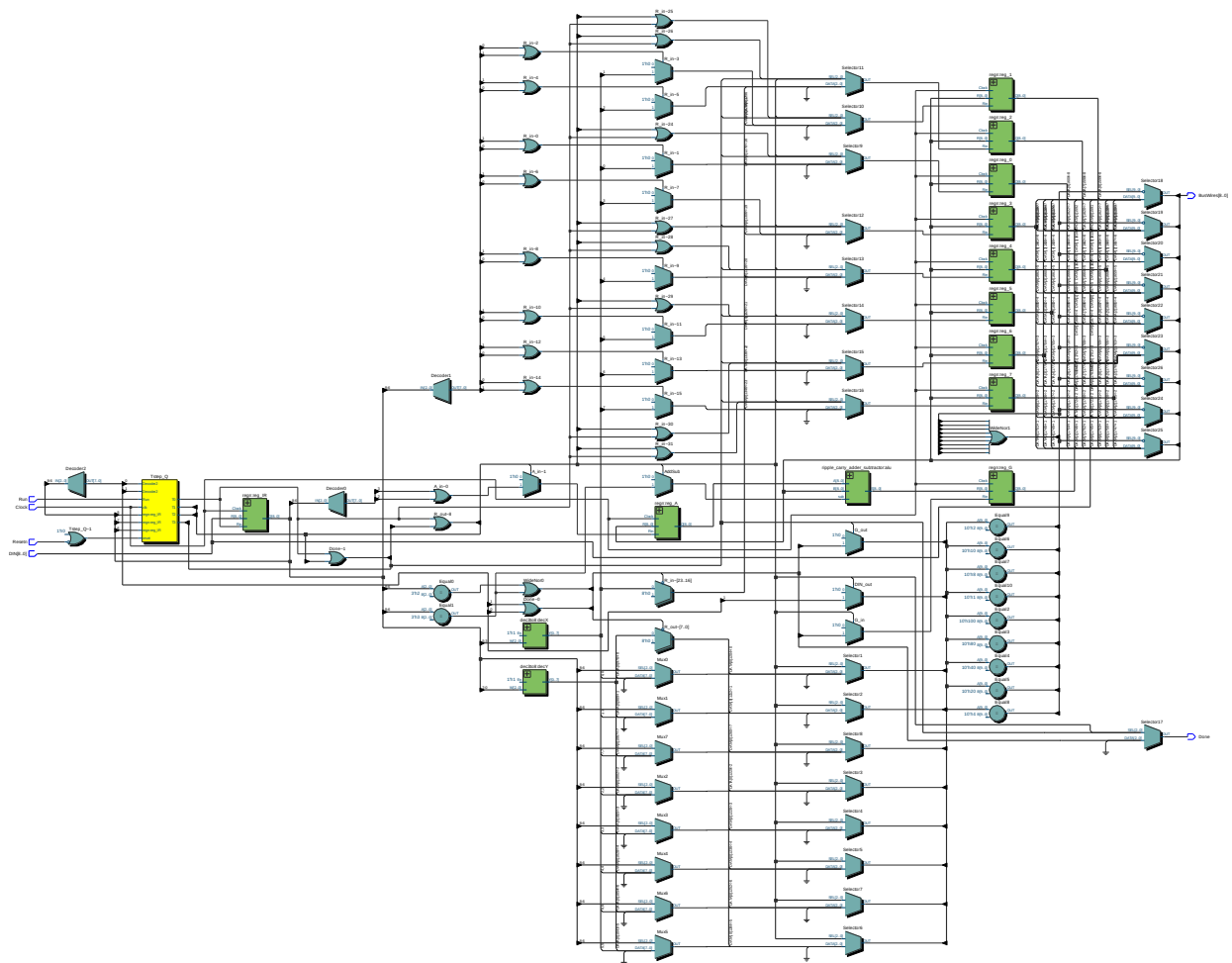
Moduł pełnego sumatora

```
module full_adder(
  input a, b, cin,
  output s, cout);

  assign s = cin ^ (a ^ b);
  assign cout = a & b | (a ^ b) & cin;
endmodule
```

Wyniki syntezy

RTL Viewer



State Machine Viewer układu sterowania

Graf stanów

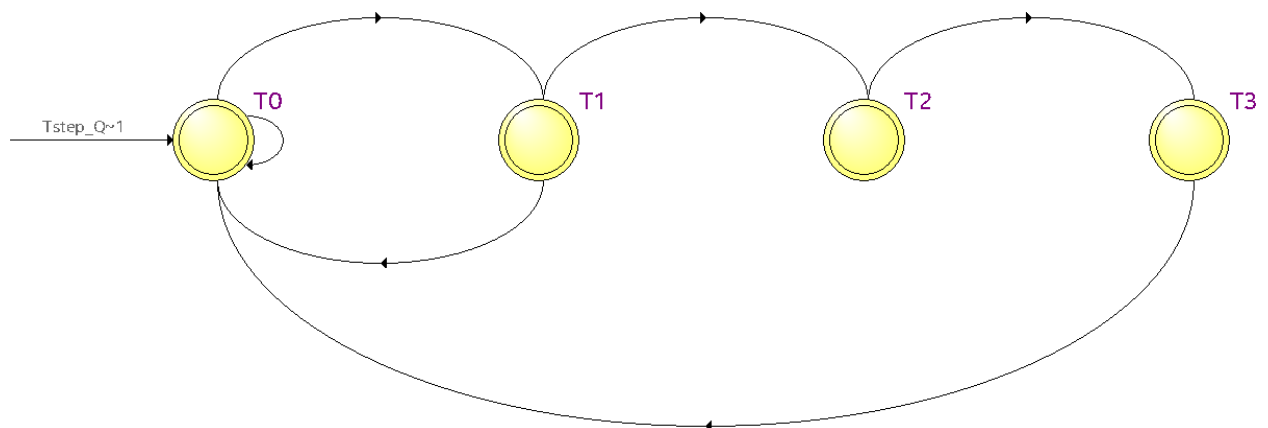


Tabela przejść

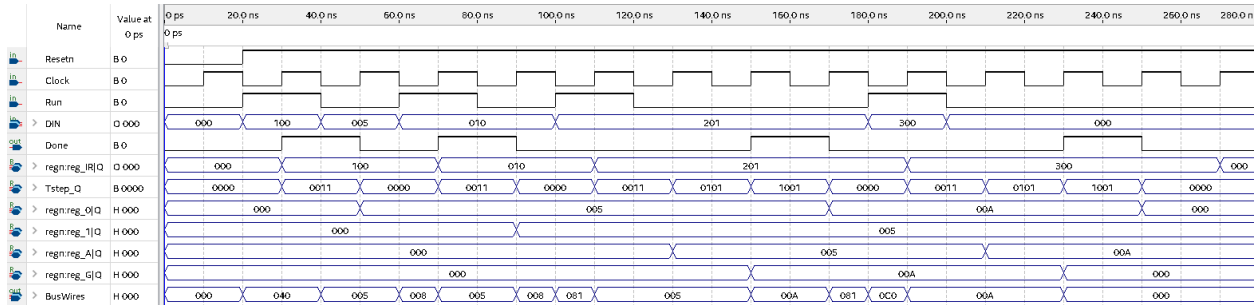
	Source State	Destination State	Condition
1	T0	T0	{!Run}
2	T0	T1	{Run}
3	T1	T2	{!Decoder2}.(!Decoder2)
4	T1	T0	{!Decoder2}.(Decoder2) + {Decoder2}
5	T2	T3	
6	T3	T0	

Tabela kodowania

	Name	T3	T2	T1	T0
1	T0	0	0	0	0
2	T1	0	0	1	1
3	T2	0	1	0	1
4	T3	1	0	0	1

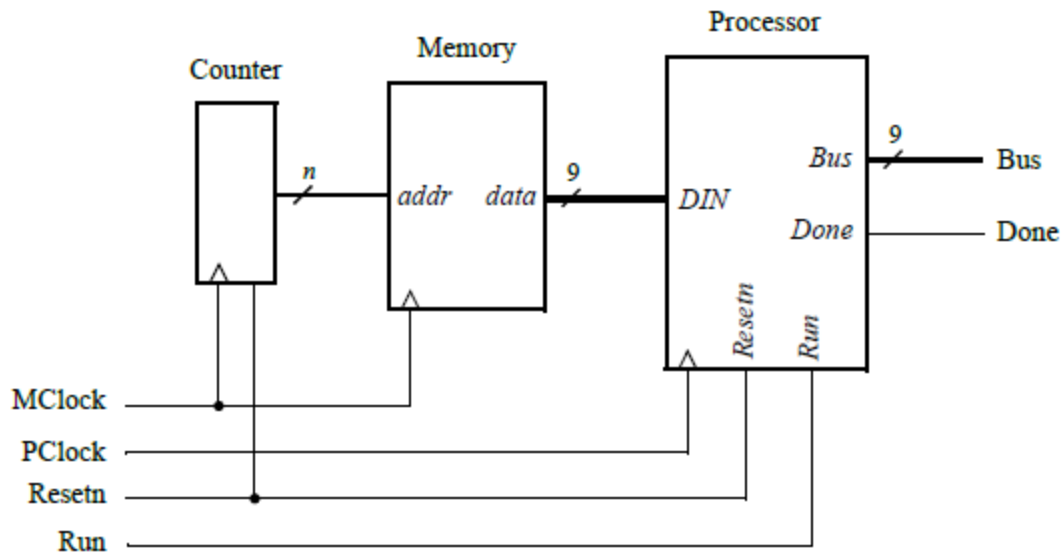
Wyniki symulacji funkcjonalnej

Wyniki pokrywają się z rysunkiem 3.



Zadanie 2

Utwórz projektor procesora processor_with_ROM, który jest podłączony do pamięci (rys. 4)



Rys. 4. Podłączenie procesora do pamięci i modułu licznika

Tutaj licznik służy do wyboru adresów komórek pamięci, z których dane wprowadzane są do procesora. Pamięć i procesor są taktowane przez różne sygnały zegarowe: MClock i PClock. Blok pamięci 32 x 9 jest zaimplementowany jako pamięć typu ROM przy użyciu funkcji bibliotecznej z IP Catalog. Przykład pliku inicjującego pamięć pokazano na rys. 5.

```
DEPTH = 32;
WIDTH = 9;
ADDRESS_RADIX = HEX;
DATA_RADIX = BIN;
CONTENT
BEGIN

00 : 001000000;    % mvi r0,#5  %
01 : 000000101;
02 : 000001000;    % mv r1,r0  %
03 : 010000001;    % add r0, r1  %
04 : 011000000;    % sub r0, r0  %
05 : 000000000;
06 : 000000000;
... (some lines not shown)
1E : 000000000;
1F : 000000000;

END;
```

Rys. 5. Przykład pliku inicjującego pamięć (MIF)

Przypisania pinów procesora:

Wyprowadzenia	Piny
Run	SW9
Reset	SW0
MClock	KEY0
PClock	KEY1
Done	LED9
Bus	LED8-0

Wykonaj symulację procesora. Upewnij się, że wyniki są takie same jak na Rys. 3.

Kod w Verilog

Moduł najwyższego poziomu z przyporządkowaniem do pinów płyty

```
module processor_with_ROM_on_board(
    input [9:0] SW,
    input [1:0] KEY,
    output [9:0] LEDR);

    processor_with_ROM proc(~KEY[0], SW[0], ~KEY[1], SW[9], LEDR[9], LEDR[8:0]);

endmodule
```

Moduł procesora pobierającego instrukcje z pamięci ROM

```
module processor_with_ROM(
    input MClock, Resetn, PClock, Run,
    output Done,
    output [8:0] Buswires);

    wire [4:0] read_addr;
    counter_N_bits #(5) address_counter(MClock, Resetn, read_addr);
    wire [8:0] DIN /* synthesis keep */;
    rom32x9 rom(read_addr, MClock, DIN);
    proc_1 proc(DIN, Resetn, PClock, Run, Done, Buswires);

endmodule
```

Moduł N-bitowego licznika

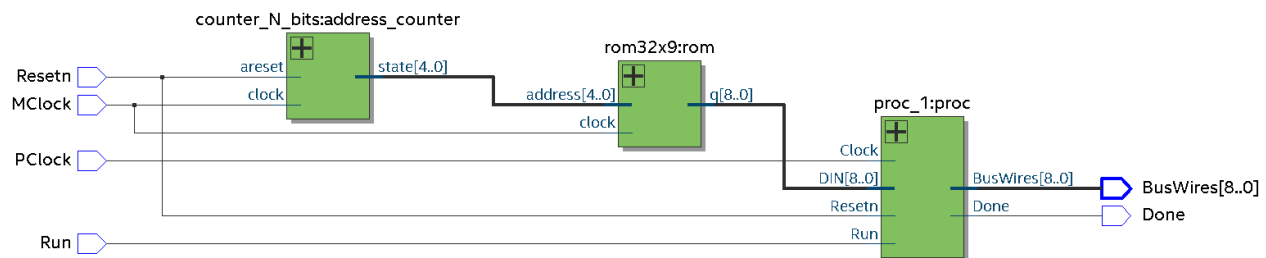
```
module counter_N_bits
    #(parameter N = 4)(
    input clock, areset,
    output reg [N-1:0] state);

    initial state = {N{1'b0}};
    always @ (posedge clock, negedge areset)
        if (~areset) state <= {N{1'b0}};
        else state <= state + 1'b1;

endmodule
```

Moduły procesora: proc_1, regn, dec3to8, ripple_carry_adder_subtractor, full_adder są takie same jak w zadaniu 1.

Wyniki syntezy



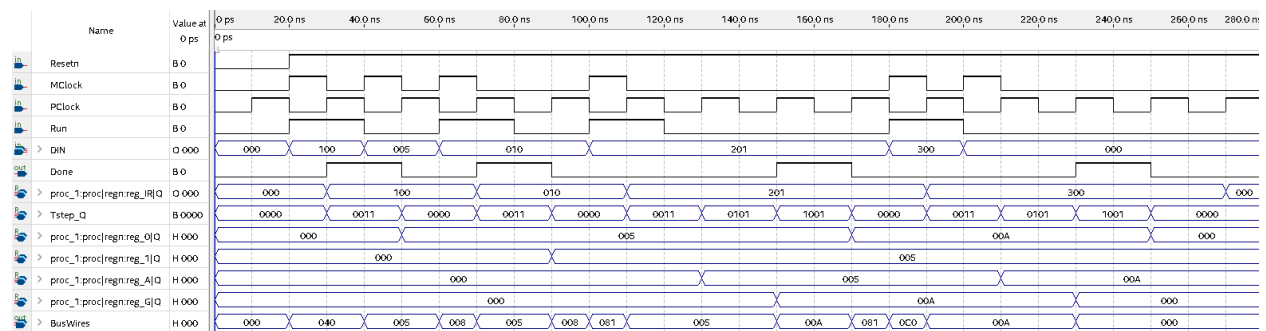
Wyniki symulacji funkcjonalnej

Przyjmujemy następującą zawartość pliku inicjującego pamięć (MIF):

```
WIDTH=9;
DEPTH=32;
ADDRESS_RADIX=HEX;
DATA_RADIX=BIN;
CONTENT BEGIN
    00 : 001000000; % mvi r0,#5 %
    01 : 000000101;
    02 : 000001000; % mv r1,r0 %
    03 : 010000001; % add r0, r1 %
    04 : 011000000; % sub r0, r0 %
    [05..1F] : 000000000;
END;
```

END;

Wyniki pokrywają się z rysunkiem 3.



Wnioski

Wszystkie zadania zostały zrealizowane.